EMSI — Développement Front-End

Séance 3 Next.js — TP : Full-Stack (Prisma + Performance + Déploiement)

Projet	TaskFlow Next — version full-stack finale
Objectif	Prisma + SQLite, next/image, next/font, loading.tsx, SSG, déploiement Vercel
Stack	Next.js 14+ + TypeScript + Prisma + SQLite
Prérequis	Séances 1-2 Next.js terminées (routing, Server Actions, API Routes, auth cookies)

Partie 1 : Installer Prisma + SQLite

1.1 Installation

```
npm install prisma@6 @prisma/client@6
npx prisma init --datasource-provider sqlite
```

Cela crée prisma/schema.prisma et un fichier .env avec DATABASE_URL.

1.2 Définir le schéma

Modifiez prisma/schema.prisma :

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model Project {
  id        Int      @id @default(autoincrement())
  name      String
  color     String   @default("#3498db")
  createdAt DateTime @default(now())
}

model User {
  id       Int    @id @default(autoincrement())
  email    String @unique
  password String
  name     String
}
```

1.3 Migrer et générer le client

```
npx prisma migrate dev --name init
# Crée les tables + génère le client TypeScript
```

1.4 Créer l'instance Prisma

Créez lib/prisma.ts :

```
import { PrismaClient } from '@prisma/client';

const globalForPrisma = globalThis as unknown as { prisma: PrismaClient };

export const prisma = globalForPrisma.prisma || new PrismaClient();

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

Le pattern globalForPrisma évite de créer une nouvelle connexion à chaque hot reload en dev.

1.5 Seed : insérer des données initiales

Créez prisma/seed.ts :

```
import { PrismaClient } from '@prisma/client';
const prisma = new PrismaClient();

async function main() {
  await prisma.user.create({
    data: { email: 'admin@taskflow.com', password: 'password123', name: 'Admin' }
  });
  await prisma.project.createMany({
    data: [
      { name: 'App Mobile', color: '#3498db' },
      { name: 'API Back', color: '#2ecc71' },
    ]
  });
  console.log('Seed done!');
}
main();
```

```
npx tsx prisma/seed.ts
```

➤ *Q1 : Run npx prisma studio. Voyez-vous les données dans les tables Project et User ?*

prisma studio : interface web → qui lit le fichier SQLite

Partie 2 : Migrer les API Routes vers Prisma

Remplacez fs.readFileSync/writeFileSync par Prisma dans vos API Routes.

2.1 app/api/projects/route.ts

```
import { NextResponse } from 'next/server';
import { prisma } from '@/lib/prisma';

export async function GET() {
  const projects = await prisma.project.findMany({
    orderBy: { createdAt: 'desc' }
  });
  return NextResponse.json(projects);
}

export async function POST(request: Request) {
  const { name, color } = await request.json();
  const project = await prisma.project.create({
    data: { name, color }
  });
  return NextResponse.json(project, { status: 201 });
}
```

➤ *Q2 : Comparez ce code avec l'ancien (fs.readFileSync + JSON.parse + push + writeFileSync). Combien de lignes en moins ?*

2.2 app/api/projects/[id]/route.ts

À VOUS : Ecrivez GET, PUT et DELETE avec Prisma au lieu de fs :

Indices :

GET : prisma.project.findUnique({ where: { id: Number(id) } })

PUT : prisma.project.update({ where: { id: Number(id) }, data: { name } })

DELETE : prisma.project.delete({ where: { id: Number(id) } })

Attention : l'id dans l'URL est un string. Prisma attend un Int. Utilisez Number(id).

2.3 Migrer les Server Actions

Modifiez app/actions/projects.ts :

```
'use server';

import { revalidatePath } from 'next/cache';
import { prisma } from '@/lib/prisma';

export async function addProject(formData: FormData) {
  const name = formData.get('name') as string;
  const color = formData.get('color') as string;
  await prisma.project.create({ data: { name, color } });
  revalidatePath('/dashboard');
}
```

À VOUS : Migrez renameProject et deleteProject vers Prisma (même pattern : prisma.project.update / .delete).

➤ *Q3 : Supprimez db.json. L'app fonctionne-t-elle toujours ? Pourquoi ?*

Partie 3 : Dashboard avec Prisma direct

Le Dashboard est un Server Component. Il peut appeler Prisma DIRECTEMENT (pas besoin de fetch API) :

```
// app/dashboard/page.tsx

import { prisma } from '@/lib/prisma';
import AddProjectForm from './AddProjectForm';
import { deleteProject } from '../actions/projects';

export default async function DashboardPage() {
  const projects = await prisma.project.findMany({
    orderBy: { createdAt: 'desc' }
  });

  return (
    <div style={{ padding: '2rem' }}>
      <h1>Dashboard</h1>
      <p>{projects.length} projets</p>
      <AddProjectForm />
      <ul>
        {projects.map(p => (
          <li key={p.id} style={{ display: 'flex', gap: 8, alignItems: 'center',
marginBottom: 8 }}>
            <span style={{ width: 12, height: 12, borderRadius: '50%', background: p.color,
display: 'inline-block' }} />
            <a href={`/projects/${p.id}`}>{p.name}</a>
            <form action={deleteProject} style={{ display: 'inline' }}>
              <input type="hidden" name="id" value={p.id} />
              <button type="submit">🗑️</button>
            </form>
          </li>
        ))}
      </ul>
    </div>
  );
}
```

Le Server Component appelle prisma directement. Pas de fetch('/api/projects'). Pas d'Axios. La BDD est lue DIRECTEMENT sur le serveur.

➤ *Q4 : Pourquoi le Server Component peut appeler prisma.project.findMany() directement mais un Client Component ('use client') ne peut PAS ?*

Partie 4 : Performance

4.1 next/image

Créez un fichier app/not-found.tsx et lancer le projet avec la balise `img` et remplacez-la par la suite avec la balise `Image`:

```
import Image from "next/image";
export default function NotFound() {
  return (
    <div style={{ padding: "2rem", textAlign: "center" }}>
      <h1 style={{ fontSize: "3rem", color: "#ccc" }}>404</h1>

      <div
        style={{ display: "flex", justifyContent: "center", margin: "2rem 0" }}
      >
        <Image
          src="/404.png"
          alt="Page not found"
          width={300}
          height={200}
          priority
        />
        {/* <img src="/404.png" alt="TaskFlow" /> */}
      </div>

      <p>Cette page n'existe pas</p>

      <a href="/dashboard" style={{ color: "#1B8C3E" }}>
        ← Retour au Dashboard
      </a>
    </div>
  );
}
```

priority = charger immédiatement (pour les images above-the-fold comme le logo).

4.2 next/font

Modifiez app/layout.tsx pour utiliser une police optimisée :

```
import { Inter } from 'next/font/google';

const inter = Inter({ subsets: ['latin'] });

export default async function RootLayout({ children }: { children: React.ReactNode }) {
  return (
```

```
    <html lang="fr">
      <body className={inter.className}>
        {/* ... Header, main, etc. */}
      </body>
    </html>
  );
}
```

➤ *Q5 : Ouvrez F12 > Network > Font. Combien de requêtes externes voyez-vous pour la police ?*

Zéro ! La police est téléchargée au build et servie localement par Next.js.

4.3 loading.tsx

Créez app/dashboard/loading.tsx :

```
export default function Loading() {
  return (
    <div style={{ padding: '2rem', textAlign: 'center' }}>
      <p>Chargement des projets...</p>
    </div>
  );
}
```

Ce fichier s'affiche AUTOMATIQUEMENT pendant que le Server Component fait son fetch Prisma.

4.4 error.tsx

À VOUS : Créez app/dashboard/error.tsx. Il doit afficher un message d'erreur et un bouton « Réessayer ».

error.tsx doit être un Client Component ('use client') car il reçoit un prop reset() pour réessayer.

```
'use client';

export default function Error({ error, reset }: { error: Error; reset: () => void }) {
  return (
    <div style={{ padding: '2rem', textAlign: 'center' }}>
      <h2>Une erreur est survenue</h2>
      <p>{error.message}</p>
      <button onClick={() => reset()}>Réessayer</button>
    </div>
  );
}
```

Partie 5 : generateStaticParams

Pré-générez les pages projets au build pour un chargement instantané :

Modifiez app/projects/[id]/page.tsx :

```
import { prisma } from '@/lib/prisma';
import { notFound } from 'next/navigation';

// Pré-générer les pages au build
export async function generateStaticParams() {
  const projects = await prisma.project.findMany();
  return projects.map(p => ({ id: String(p.id) }));
}

export default async function ProjectPage({ params }: { params: Promise<{ id: string }> }) {
  const { id } = await params;
  const project = await prisma.project.findUnique({
    where: { id: Number(id) }
  });

  if (!project) notFound();

  return (
    <div style={{ padding: '2rem' }}>
      <h1>
        <span style={{ display: 'inline-block', width: 16, height: 16,
          borderRadius: '50%', background: project.color, marginRight: 8 }} />
        {project.name}
      </h1>
      <p>Créé le : {project.createdAt.toLocaleDateString('fr-FR')}</p>
      <a href="/dashboard">← Retour</a>
    </div>
  );
}
```

➤ *Q6 : Avec generateStaticParams, les pages /projects/1 et /projects/2 sont générées quand ? Au build ou à chaque requête ?*

➤ *Q7 : Si un nouveau projet est créé après le build, la page /projects/3 existe-t-elle ?*

Par défaut oui, Next.js génère les nouvelles pages à la demande (fallback dynamique). Mais elles ne seront pas aussi rapides que les pages pré-générées au build.

Partie 6 : Déploiement

6.1 Préparer le projet

```
# Vérifier que le build fonctionne localement
npm run build
npm start
# Tester http://localhost:3000
```

Si le build échoue, corrigez les erreurs TypeScript avant de déployer.

6.2 Push sur GitHub

```
# Créer un .gitignore si pas fait
# Vérifier que .env.local et node_modules sont ignorés

git init
git add .
git commit -m 'TaskFlow Next.js full-stack'
git remote add origin https://github.com/votre-user/taskflow-next.git
git push -u origin main
```

6.3 Déployer sur Vercel

1. Allez sur vercel.com et connectez-vous avec GitHub
2. Cliquez « Add New Project » → importez taskflow-next
3. Ajoutez la variable DATABASE_URL dans Environment Variables
4. Cliquez Deploy

➤ *Q8 : Votre app est déployée ! Quelle est l'URL ? Testez le login, la création de projet, la navigation.*

SQLite ne fonctionne pas sur Vercel en production (filesystem read-only). Pour une vraie production, utilisez PostgreSQL (Vercel Postgres ou Supabase). Mais pour le TP, le build + preview suffisent.

Partie 7 : Comparaison finale

➢ *Q9 : Remplissez ce tableau :*

	React SPA (Vite)	Next.js Full-Stack
Routing		
Data fetching		
Mutations (CRUD)		
Auth		
Base de données		
Déploiement		
SEO		
Performance		
Nombre de projets		

➢ *Q10 : Si vous deviez créer une startup demain, choisiriez-vous React SPA + Express ou Next.js full-stack ? Pourquoi ?*

Structure finale TaskFlow Next.js

```
taskflow-next/
  app/
  ├── layout.tsx               // next/font + cookies() + Header + LogoutButton
  ├── page.tsx                 // accueil
  ├── not-found.tsx            // 404 globale
  ├── login/page.tsx           // 'use client' + useActionState
  ├── dashboard/
  │   ├── page.tsx              // Server Component + prisma direct
  │   ├── loading.tsx           // Suspense automatique
  │   ├── error.tsx             // ErrorBoundary automatique
  │   └── AddProjectForm.tsx   // 'use client' + useFormStatus
  ├── projects/[id]/
  │   ├── page.tsx              // prisma + generateStaticParams
  │   └── not-found.tsx         // 404 spécifique
  ├── actions/
  │   ├── projects.ts           // 'use server' + prisma CRUD
  │   └── auth.ts               // 'use server' + cookies
  ├── api/projects/
  │   ├── route.ts              // GET + POST (prisma)
  │   └── [id]/route.ts         // GET + PUT + DELETE (prisma)
  └── components/LogoutButton.tsx

  lib/prisma.ts                // PrismaClient singleton
  prisma/
  ├── schema.prisma             // modèles Project + User
  ├── seed.ts                   // données initiales
  └── dev.db                    // BDD SQLite
  middleware.ts                 // protection routes
  .env.local                    // DATABASE_URL
```